

PLAN DE COURS : SOLUTIONS DE CALCUL AWS (COMPUTATION)

- **Code de programme** : – Module 6 Service de Calcul AWS— Spécialisation Cloud
- **Poids horaire** : 45 heures (15 semaines × 3 heures)
- **Approche pédagogique** : 30 % Théorie et architecture, 70 % Laboratoires pratiques et gestion de coûts

DESCRIPTIF DU COURS

Ce module spécialisé permet à l'étudiant de maîtriser, de comparer et de déployer l'éventail des services de calcul (*Compute*) offerts par Amazon Web Services (AWS). Allant des modèles traditionnels d'infrastructure brute aux paradigmes modernes de l'infonuagique, le cours explore le IaaS (EC2), l'orchestration de conteneurs (ECS/ECR/Fargate), le PaaS (Elastic Beanstalk), le Serverless (Lambda avec Java) et le calcul de masse (AWS Batch). L'étudiant sera outillé pour sélectionner le bon modèle de calcul selon les critères de performance, de haute disponibilité, de complexité opérationnelle et d'optimisation financière.

STRUCTURE DES SEMAINES ET ÉVOLUTION DU LABORATOIRE

Bloc 1 : Modèle IaaS — Contrôle total de l'infrastructure (12 heures)

Semaine 1 : Introduction aux modèles de calcul AWS et Fondations EC2

- **Théorie** : Taxonomie des services de calcul AWS (IaaS, PaaS, Serverless, Container, Batch). Introduction approfondie à Amazon EC2 (*Elastic Compute Cloud*). Choix des types d'instances (Général, Calcul, Mémoire) et des types de stockage (EBS).
- **Notes Enseignant** : Présenter la matrice de responsabilité partagée d'AWS dédiée au modèle IaaS.

Semaine 2 : Sécurisation et connectivité d'une instance

- **Théorie** : Rôle des paires de clés SSH/RDP. Mécanisme de pare-feu virtuel avec les **Security Groups** (règles d'entrée et de sortie). Concepts d'adresses IP publiques, privées et Elastic IPs.
- **Démonstration** : Déploiement guidé d'une instance Linux et d'une instance Windows en direct.

Semaine 3 : LABORATOIRE 1 — Déploiement et diagnostic EC2

- **Laboratoire Pratique** : Création et configuration d'une instance EC2 Linux. Paramétrage d'un Security Group pour autoriser le trafic SSH et HTTP. Connexion distante en CLI. Installation d'un serveur Web simple.
- **Finances & Nettoyage** : Analyse des métriques de CPU via CloudWatch et exécution de la procédure d'arrêt définitif (*Terminate*) pour éviter le gaspillage budgétaire.

Semaine 4 : Transition vers la conteneurisation d'applications

- **Théorie** : Pourquoi passer du serveur virtuel (EC2) au conteneur ? Anatomie d'un fichier Dockerfile. Rôle d'**Amazon ECR** (*Elastic Container Registry*) pour le stockage sécurisé des images.

Bloc 2 : Modèle Container & Orchestration (9 heures)

Semaine 5 : Architecture Amazon ECS et Microservices

- **Théorie** : Fonctionnement d'**Amazon ECS** (*Elastic Container Service*). Concepts clés : *Task Definitions* (blueprints), *Services* (scalabilité/exécution) et *Clusters*. Comparaison entre le modèle EC2 traditionnel et l'architecture Serverless **AWS Fargate**.
- **Notes Enseignant** : Utiliser un diagramme d'architecture pour montrer la séparation entre la couche de gestion (ECS) et la couche d'exécution (Fargate).

Semaine 6 : LABORATOIRE 2 — Orchestration de conteneurs avec ECS/Fargate

- **Laboratoire Pratique** : Création d'une image Docker simple sur le poste local. Connexion et téléversement (*Push*) de l'image vers un dépôt Amazon ECR. Création d'un cluster ECS. Déploiement automatisé d'un service web à l'aide d'AWS Fargate. Validation de l'accès public.
- **Nettoyage** : Suppression méthodique du service, du cluster et du dépôt ECR.

Semaine 7 : Examen Intra-trimestriel & Détection de plateforme

- **Évaluation (1.5 h)** : Quiz conceptuel et technique sur l'architecture EC2 et le modèle de conteneurisation AWS.
- **Théorie (1.5 h)** : Introduction au concept de PaaS (*Platform as a Service*) avec **AWS Elastic Beanstalk**. Analyse de la détection automatique de plateformes (Java, Python, Node.js).

Bloc 3 : Modèle PaaS & Serverless (12 heures)

Semaine 8 : Déploiement applicatif simplifié avec Elastic Beanstalk

- **Théorie** : Gestion du cycle de vie des applications web sans gestion de l'infrastructure sous-jacente. Différence stratégique entre un redéploiement complet et une mise à jour progressive (**Rolling Update**).
- **Notes Enseignant** : Montrer dans la console AWS les ressources réelles créées en arrière-plan par Beanstalk (EC2, Auto Scaling, CloudWatch).

Semaine 9 : LABORATOIRE 3 — Déploiement d'une Application Web (PaaS)

- **Laboratoire Pratique** : Création d'un environnement AWS Elastic Beanstalk. Déploiement d'une application web Java d'exemple. Vérification et validation de l'URL publique. Modification du code source et exécution d'un *Rolling Update* pour déployer la nouvelle version en direct.
- **Nettoyage** : Destruction complète de l'environnement et de l'application Beanstalk.

Semaine 10 : Le Paradigme Serverless et l'Architecture Événementielle

- **Théorie** : Principes du Serverless avec **AWS Lambda**. Fonctionnement par déclencheurs (*Triggers*). Structure du cycle de vie d'une fonction Lambda. Concept de tarification à la milliseconde d'exécution.

Semaine 11 : Développement de Fonctions Lambda en Java

- **Théorie** : Structure d'un projet Java pour AWS Lambda à l'aide de Maven ou Gradle. Implémentation de l'interface et du Handler d'événements standard : `handleRequest(Map<String,Object> input, Context context)`.
- **Laboratoire Pratique (Partie A)** : Configuration de l'environnement de développement local et écriture du code Java.

Bloc 4 : Événements, Calcul de masse et Synthèse (12 heures)

Semaine 12 : LABORATOIRE 4 — AWS Lambda déclenché par stockage (S3)

- **Laboratoire Pratique (Partie B)** : Compilation et déploiement du package JAR Java sur AWS Lambda. Configuration d'un déclencheur automatique lié à un compartiment **Amazon S3** (téléversement de fichier). Validation de l'exécution et débogage du Handler en analysant les journaux dans **Amazon CloudWatch Logs**.
- **Nettoyage** : Suppression de la fonction Lambda, du déclencheur et du compartiment S3 de test.

Semaine 13 : Calcul de masse et traitement par lots avec AWS Batch

- **Théorie** : Cas d'usage pour les calculs longs, lourds et non interactifs. Composants d'**AWS Batch** : *Compute Environments* (choix des instances managées/Spot), *Job Queues* (priorisation) et *Job Definitions* (paramètres d'exécution).

Semaine 14 : LABORATOIRE 5 — Planification de Jobs massifs avec AWS Batch

- **Laboratoire Pratique** : Configuration d'un environnement de calcul et d'une file d'attente (*Job Queue*). Définition d'un job Batch s'appuyant sur une commande d'affichage ou un script simple. Soumission du job à la file d'attente. Suivi des statuts de transition (*SUBMITTED*, *RUNNABLE*, *RUNNING*, *SUCCEEDED*) et lecture des logs dans CloudWatch.
- **Nettoyage** : Désactivation et suppression du job, de la file d'attente et de l'environnement de calcul.

Semaine 15 : Évaluation finale et arbitrage architectural

- **Examen final pratique (3 heures)** : Scénario d'entreprise concret fourni à l'étudiant. Celui-ci doit analyser un besoin d'affaires, justifier l'arbitrage architectural entre **EC2 vs Beanstalk vs Lambda vs Batch**, puis implanter et valider la solution de calcul sélectionnée sur la console AWS dans le respect des contraintes budgétaires.

ENVIRONNEMENT MATÉRIEL ET LOGICIEL REQUIS

- **Accès Cloud** : Compte d'infrastructure AWS (ou environnement fourni par AWS Academy) doté des privilèges IAM nécessaires pour manipuler les ressources de calcul.
- **Environnement local (Poste étudiant)** :
 - IDE Java (IntelliJ IDEA, Eclipse ou VS Code).
 - Outils de build Java : Apache Maven ou Gradle.

- Client SSH de terminal et Docker Desktop pour la création d'images locales.